

BLG462 - Büyük Veri Dönem Projesi

Twitter Verisi ile Uçtan Uca Büyük Veri Pipeline'ı Proje Raporu

Hazırlayan: Berat Turan

Öğrenci Numarası: 221201018

GİRİŞ VE PROJE AMACI

Bu projede, Kaggle platformundan temin edilen gerçek bir "Twitter US Airline Sentiment" veri seti kullanılarak uçtan uca bir büyük veri işleme ardışık düzeni (pipeline) tasarlanmıştır. Projenin amacı; ham Twitter verisini Python ve Pandas kütüphaneleriyle temizlemek, MongoDB veritabanında modellemek, PySpark ile dağıtık işleme adımlarını uygulamak ve tüm bu süreci ETL perspektifinden görselleştirerek belgelemektir.

AŞAMA 1: VERİ TEMİNİ VE KEŞİF

1.1. Veri Seti Hakkında

Kaggle platformundan indirilen veri seti, havayolu şirketlerine atılan yaklaşık 14.000 tweetten ve 15 sütundan oluşmaktadır.

1.2. DIKW Piramidi Çerçevesinde Veri Setinin Değerlendirilmesi

- **Data (Veri):** 14.000 civarında ham tweet metni, atılma tarihleri ve havayolu şirket isimleri.
- **Information (Bilgi):** Bu tweetlerin pozitif, negatif veya nötr olarak etiketlenmesi; negatif olanların "rötar, kayıp bagaj, kötü hizmet" gibi alt kategorilere ayrılması.
- **Knowledge (Bilgi Birikimi):** Hangi havayolu şirketinin en çok hangi konudan şikayet aldığının ve bu müşteri memnuniyetsizliklerinin haftanın veya yılın hangi dönemlerinde yoğunlaştığının tespit edilmesi.
- **Wisdom (Bilgelik):** Şikayetlerin kök nedenlerine yönelik operasyonel iyileştirmeler yapılması (örn. bagaj takip sistemlerinin yenilenmesi) ve yolcu deneyimini artıracak stratejik kararların alınması.

1.3. Veri Keşfi ve Temel İstatistikler

Pandas ile okutulan veri setinin ilk boyutları ve istatistiksel dağılımı aşağıdaki gibidir:

```

-- Toplam Satır ve Sütun Sayısı: (14640, 15)

Sütun İsimleri:
['tweet_id', 'airline_sentiment', 'airline_sentiment_confidence', 'negativereason', 'negativereason_confidence', 'airline', 'airline_sentiment_gold', 'name', 'negativereason_gold', 'retweet_count', 'text', 'tweet_coord', 'tweet_created', 'tweet_location', 'user_timezone']

tweet_id airline_sentiment airline_sentiment_confidence negativereason negativereason_confidence airline airline_sentiment_gold name negativereason_gold retweet_count text tweet_coord tweet_created tweet_location user_timezone
0 57030613367776513 neutral 1.0000 NaN NaN Virgin America NaN cairin NaN 0 @VirginAmerica What @dhepburn said NaN 2015-02-24 11:35:52 -0800 NaN Eastern Time (US & Canada)
1 57030113088122368 positive 0.3486 NaN 0.0000 Virgin America NaN jrandino NaN 0 @VirginAmerica plus you've added commercials L NaN 2015-02-24 11:15:59 -0800 NaN Pacific Time (US & Canada)
2 570301083672613571 neutral 0.6837 NaN NaN Virgin America NaN yonnalynn NaN 0 @VirginAmerica I don't today... Must mean I n. NaN 2015-02-24 11:15:40 -0800 NaN Central Time (US & Canada)
3 570301031407624196 negative 1.0000 Bad Flight 0.7033 Virgin America NaN jrandino NaN 0 @VirginAmerica it's really aggressive to blast NaN 2015-02-24 11:15:36 -0800 NaN Pacific Time (US & Canada)
4 570300617074452722 negative 1.0000 Can't Tell 1.0000 Virgin America NaN jrandino NaN 0 @VirginAmerica and it's a really big bad thing NaN 2015-02-24 11:14:45 -0800 NaN Pacific Time (US & Canada)

--- Veri Tipleri ve Eksik (Null) Değer Durumu ---
<class 'pandas.DataFrame'>
RangeIndex: 14640 entries, 0 to 14639
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   tweet_id              14640 non-null   int64
1   airline_sentiment      14640 non-null   str
2   airline_sentiment_confidence  14640 non-null   float64
3   negativereason         9178 non-null   str
4   negativereason_confidence  10522 non-null   float64
5   airline                14640 non-null   str
6   airline_sentiment_gold  48 non-null      str
7   name                  14640 non-null   str
8   negativereason_gold    32 non-null      str
9   retweet_count          14640 non-null   int64
10  text                  14640 non-null   str
11  tweet_coord            1019 non-null    str
12  tweet_created          14640 non-null   str
13  tweet_location         9907 non-null    str
14  user_timezone          8630 non-null    str
dtypes: float64(2), int64(2), str(11)
memory usage: 1.7 MB

--- Temel İstatistikler ---

tweet_id airline_sentiment airline_sentiment_confidence negativereason negativereason_confidence airline airline_sentiment_gold name negativereason_gold retweet_count text tweet_coord tweet_created tweet_location user_timezone
count 1.464000e+04 14640 14640.000000 9178 10522.000000 14640 40 14640 32 14640.000000 14640 1019 14640 9907 8620
unique NaN 3 NaN 10 NaN 6 3 7701 13 NaN 14427 832 14247 3081 85
top NaN negative NaN Customer Service Issue NaN United negative JetBlueNews Customer Service Issue NaN @united thanks [0.0, 0.0] 2015-02-24 09:54:34 -0800 Boston, MA Eastern Time (US & Canada)
freq NaN 9178 NaN 2910 NaN 3822 32 63 12 NaN 6 164 5 157 3744
mean 5.682184e+17 NaN 0.900169 NaN 0.638298 NaN NaN NaN NaN 0.082650 NaN NaN NaN NaN NaN
std 7.79112e+14 NaN 0.162030 NaN 0.330440 NaN NaN NaN NaN 0.745778 NaN NaN NaN NaN NaN
min 5.675883e+17 NaN 0.335000 NaN 0.000000 NaN NaN NaN NaN 0.000000 NaN NaN NaN NaN NaN
25% 5.68592e+17 NaN 0.682300 NaN 0.360600 NaN NaN NaN NaN 0.000000 NaN NaN NaN NaN NaN
50% 5.694779e+17 NaN 1.000000 NaN 0.670600 NaN NaN NaN NaN 0.000000 NaN NaN NaN NaN NaN
75% 5.698905e+17 NaN 1.000000 NaN 1.000000 NaN NaN NaN NaN 0.000000 NaN NaN NaN NaN NaN
max 5.703109e+17 NaN 1.000000 NaN 1.000000 NaN NaN NaN NaN 44.000000 NaN NaN NaN NaN NaN

```

AŞAMA 2: VERİ TEMİZLEME VE HAZIRLIK

2.1. Temizleme İşlemleri

Python Regex ve Pandas kullanılarak aşağıdaki temizlik adımları uygulanmıştır:

- Metin içerisindeki URL'ler, mention'lar (@kullanıcı) ve özel karakterler temizlenmiştir.
- Hashtag'lerin (#) sadece işaretleri silinmiş, kelime kökleri veride bırakılmıştır.
- Veri setindeki tekrarlayan (duplicate) kayıtlar tespit edilip silinmiştir.
- negativereason sütunu başta olmak üzere eksik (Null/NaN) veriler mantıksal değerlerle ("Belirtilmedi", "Bilinmiyor") doldurulmuştur.
- Metin tabanlı tweet_created sütunu standart datetime tipine dönüştürülmüştür.

```
# Temizleme öncesi kayıt sayısını tutalım
baslangic_kayit = len(df)
print(f"Temizleme öncesi toplam kayıt: {baslangic_kayit}")

def metin_temizle(text):
    if type(text) != str:
        return text

    # 1. URL'leri temizleme
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)
    # 2. Mention (@kullanici) temizleme
    text = re.sub(r'@\w+', '', text)
    # 3. Hashtag (#) işaretini silme (kelime kalıyor)
    text = re.sub(r'#', '', text)
    # 4. Özel karakterleri temizleme (Sadece harf, rakam ve boşluklar kalsın)
    text = re.sub(r'^\w\s', '', text)

    # Fazla boşlukları alıp küçük harfe çevirelim
    text = " ".join(text.split()).lower()
    return text

# Fonksiyonu uygulayıp yeni sütun oluşturalım
df['cleaned_text'] = df['text'].apply(metin_temizle)

# Temizlenmiş haliyle eski halini karşılaştıralım
df[['text', 'cleaned_text']].head()
```

... Temizleme öncesi toplam kayıt: 14640

	text	cleaned_text
0	@VirginAmerica What @dhepburn said.	what said
1	@VirginAmerica plus you've added commercials t...	plus youve added commercials to the experience...
2	@VirginAmerica I didn't today... Must mean I n...	i didnt today must mean i need to take another...
3	@VirginAmerica it's really aggressive to blast...	its really aggressive to blast obnoxious enter...
4	@VirginAmerica and it's a really big bad thing...	and its a really big bad thing about it

```
# Tekrarlanan (duplicate) kayıtları silme
df = df.drop_duplicates()

# Eksik (null) değerlerin tespiti ve giderilmesi
# En önemli boşluk 'negativereason' sütununda. Orayı dolduruyoruz.
df['negativereason'] = df['negativereason'].fillna('Belirtilmedi')

# Kullanılmayacak ama null olan diğer sütunları da standart bir değerle dolduralım
df['airline_sentiment_gold'] = df['airline_sentiment_gold'].fillna('Bilinmiyor')
df['negativereason_gold'] = df['negativereason_gold'].fillna('Bilinmiyor')
df['tweet_coord'] = df['tweet_coord'].fillna('Bilinmiyor')

# Sütun veri tiplerinin doğrulanması (tarihi datetime yapıyoruz)
df['tweet_created'] = pd.to_datetime(df['tweet_created'])

# Temizleme sonrası durumu raporlama
bitis_kayit = len(df)
print(f"\nTemizleme sonrası toplam kayıt: {bitis_kayit}")
print(f"Silinen tekrarlı (duplicate) kayıt sayısı: {baslangic_kayit - bitis_kayit}")

...
Temizleme sonrası toplam kayıt: 14604
Silinen tekrarlı (duplicate) kayıt sayısı: 36
```

2.2. Veri Kalitesi Değerlendirmesi

- **Doğruluk / Tamlık (Completeness):** Veri setindeki eksik değerler iş kurallarına uygun doldurularak veri kaybı yaşanmadan %100 tamlık sağlanmıştır.
- **Tutarlılık (Consistency):** Aynı ID'ye sahip veya tamamen tekrarlayan mükerrer satırlar kaldırılarak veri tekilliği garanti altına alınmıştır.
- **Geçerlilik (Validity):** Tüm metinler gereksiz karakterlerden arındırılmış, string tipindeki tarih verileri geçersiz formatlardan kurtararak analitik sistemlerin okuyabileceği geçerli DateTime objelerine dönüştürülmüştür.

AŞAMA 3: MONGODB'YE YÜKLEME VE NOSQL SORGULARI

3.1. MongoDB Şema Tasarımı

Projede ilişkisel bir veritabanı yerine MongoDB kullanıldığı için veri **Düz Doküman** mimarisinde modellenmiştir. Tweet metni, havayolu şirketi, zaman damgası ve duygu analizi sonuçları tek bir JSON/BSON dokümanında tutulmuştur. "Join" işlemleri gerektiren bir ilişki olmadığı için bu düz yapı, okuma ve sorgulama performansını maksimize etmiştir.

3.2. Toplu Yükleme (Bulk Insert) ve Sorgulamalar

Temizlenen veriler PyMongo aracılığıyla twitter_db veritabanındaki tweetler koleksiyonuna tek seferde aktarılmıştır. Ardından sistem üzerinde 12 farklı işlem gerçekleştirilmiştir:

- **Temel find() Sorguları:** Belirli havayollarına ait filtrelemeler ve projeksiyon işlemleri.
- **Aggregation Pipeline:** Havayolu bazlı grup sayımları (\$group) ve ortalama güven skorlarının hesaplanması.
- **İndeksleme (Indexing):** Arama performansını artırmak için airline alanına indeks atılması ve explain() ile doğrulanması.
- **Update ve Delete:** updateOne ve updateMany ile yeni alanların eklenmesi, güven skoru çok düşük olan anlamsız kayıtların deleteMany ile temizlenmesi.
- **İleri Seviye Sorgular:** Metin içinde kelime aramak için Regex kullanımı ve birden fazla havayolunu eşleştirmek için Array (\$in) operatörünün kullanımı.

```
import pandas as pd
from pymongo import MongoClient

# 1. Temizlenmiş veriyi okuyalım
df = pd.read_csv('/content/drive/MyDrive/Buyuk_Veri_Donem_Projesi/twitter_big_data_pipeline/data/processed/cleaned_tweets.csv')

# 2. MongoDB'ye bağlanma (Yerel kurulum varsayılan port)
try:
    client = MongoClient('mongodb://localhost:27017/')
    print("MongoDB bağlantısı başarılı!")

    # Veritabanı ve koleksiyon oluşturma
    db = client['twitter_db']
    collection = db['tweetler']

    # Eğer koleksiyonda eski veri varsa temizleyelim (kodu tekrar çalıştırsak veri çifte kayıt olmasın)
    collection.delete_many({})

    # 3. Pandas DataFrame'i sözlük (dictionary) listesine çevirip MongoDB'ye toplu yükleme (bulk insert)
    records = df.to_dict('records')
    collection.insert_many(records)
    print(f"{len(records)} adet doküman MongoDB'ye başarıyla yüklendi.")

except Exception as e:
    print(f"Bir hata oluştu: {e}")
```

MongoDB bağlantısı başarılı!
14604 adet doküman MongoDB'ye başarıyla yüklendi.

```
print("--- 1. find() Sorguları ---")

# Sorgu 1: Sadece Virgin America şirketine ait pozitif tweetleri bulma (Koşullu filtreleme ve Limit)
print("\nSorgu 1: Virgin America'ya ait 2 pozitif tweet:")
sorgu1 = collection.find(
    {'airline': 'Virgin America', 'airline_sentiment': 'positive'},
    {'text': 1, 'airline_sentiment': 1, '_id': 0} # Projeksiyon: Sadece metin ve duygu gelsin
).limit(2)
for doc in sorgu1:
    print(doc)

# Sorgu 2: Güven skoru (confidence) 0.9'dan büyük olan negatif tweetler (Operatör kullanımı)
print("\nSorgu 2: Güven skoru çok yüksek olan negatif 2 tweet:")
sorgu2 = collection.find(
    {'airline_sentiment': 'negative', 'airline_sentiment_confidence': {'$gt': 0.9}},
    {'airline': 1, 'negativereason': 1, '_id': 0}
).limit(2)
for doc in sorgu2:
    print(doc)

# Sorgu 3: Belirli alanları atlayarak (skip) veri getirme
print("\nSorgu 3: İlk 5 kaydı atlayıp (skip) sonraki 1 kaydı getirme:")
sorgu3 = collection.find({}, {'text': 1, '_id': 0}).skip(5).limit(1)
for doc in sorgu3:
    print(doc)
```

... --- 1. find() Sorguları ---

Sorgu 1: Virgin America'ya ait 2 pozitif tweet:
{'airline_sentiment': 'positive', 'text': '@VirginAmerica plus you've added commercials to the experience... tacky.'}
{'airline_sentiment': 'positive', 'text': '@VirginAmerica yes, nearly every time I fly VX this â\x80\x9cx9cear wormâ\x80\x9dx9d wonâ\x80\x99t go away :)}'

Sorgu 2: Güven skoru çok yüksek olan negatif 2 tweet:
{'negativereason': 'Bad Flight', 'airline': 'Virgin America'}
{'negativereason': 'Can't Tell', 'airline': 'Virgin America'}

Sorgu 3: İlk 5 kaydı atlayıp (skip) sonraki 1 kaydı getirme:
{'text': '@VirginAmerica seriously would pay \$30 a flight for seats that didn't have this playing.\nit's really the only bad thing about flying VA'}

```

print("--- 2. Aggregation Pipeline Sorguları ---")

# Sorgu 4: Havayolu şirketlerine göre toplam tweet sayısını bulma ($group)
print("\nSorgu 4: Şirket bazlı tweet sayıları:")
pipeline1 = [
    {"$group": {"_id": "$airline", "toplam_tweet": {"$sum": 1}}},
    {"$sort": {"toplam_tweet": -1}} # Büyükten küçüğe sırala
]
for doc in collection.aggregate(pipeline1):
    print(doc)

# Sorgu 5: Sadece negatif tweetleri filtreleyip, şikayet sebeplerine göre gruptama ($match, $group)
print("\nSorgu 5: En çok rastlanan negatif tweet sebepleri:")
pipeline2 = [
    {"$match": {"airline_sentiment": "negative"}},
    {"$group": {"_id": "$negativereason", "sebep_sayisi": {"$sum": 1}}},
    {"$sort": {"sebep_sayisi": -1}},
    {"$limit": 3} # Sadece en yüksek 3 sebebi getir
]
for doc in collection.aggregate(pipeline2):
    print(doc)

# Sorgu 6: Duygu durumuna göre ortalama güven skorlarını (confidence) bulma
print("\nSorgu 6: Duygu durumuna göre ortalama güven skoru:")
pipeline3 = [
    {"$group": {
        "_id": "$airline_sentiment",
        "ortalama_guven": {"$avg": "$airline_sentiment_confidence"}
    }}
]
for doc in collection.aggregate(pipeline3):
    print(doc)

```

... --- 2. Aggregation Pipeline Sorguları ---

```

Sorgu 4: Şirket bazlı tweet sayıları:
{'_id': 'United', 'toplam_tweet': 3822}
{'_id': 'US Airways', 'toplam_tweet': 2913}
{'_id': 'American', 'toplam_tweet': 2723}
{'_id': 'Southwest', 'toplam_tweet': 2420}
{'_id': 'Delta', 'toplam_tweet': 2222}
{'_id': 'Virgin America', 'toplam_tweet': 504}

Sorgu 5: En çok rastlanan negatif tweet sebepleri:
{'_id': 'Customer Service Issue', 'sebep_sayisi': 2904}
{'_id': 'Late Flight', 'sebep_sayisi': 1660}
{'_id': 'Can't Tell', 'sebep_sayisi': 1190}

Sorgu 6: Duygu durumuna göre ortalama güven skoru:
{'_id': 'negative', 'ortalama_guven': 0.9332270881100557}
{'_id': 'neutral', 'ortalama_guven': 0.8228460368812682}
{'_id': 'positive', 'ortalama_guven': 0.8715497026338148}

```

```

print("--- 3. İndeks, Update ve Delete İşlemleri ---")

# Sorgu 7: İndeks Oluşturma ve explain() testi
print("\nSorgu 7: 'airline' alanı için indeks oluşturuluyor...")
collection.create_index([("airline", 1)])
# İndeksin çalıştığını kanıtlamak için explain kullanımı
aciklama = collection.find({"airline": "Delta"}).explain()
print("Kullanılan indeks:", aciklama['queryPlanner']['winningPlan']['inputStage']['indexName'])

# Sorgu 8: updateOne() ile tek kayıt güncelleme ($set operatörü)
print("\nSorgu 8: İlk bulunan kayda 'incelendi: true' alanı ekleniyor...")
collection.update_one(
    {"airline_sentiment": "neutral"},
    {"$set": {"incelendi": True}}
)
ornek_update1 = collection.find_one({"incelendi": True}, {"text": 1, "incelendi": 1, "_id": 0})
print("Güncellenen kayıt:", ornek_update1)

# Sorgu 9: updateMany() ile çoklu kayıt güncelleme ($set kullanımı)
print("\nSorgu 9: Retweet sayısı 0 olanlara yeni bir alan ekleniyor...")
collection.update_many(
    {"retweet_count": 0},
    {"$set": {"etkilesim_seviyesi": "dusuk"}}
)
print("Çoklu güncelleme tamamlandı.")

# Sorgu 10: deleteMany() ile kayıt silme
print("\nSorgu 10: Güven skoru (confidence) 0.3'ten küçük olan düşük kaliteli veriler siliniyor...")
silme_sonucu = collection.delete_many({"airline_sentiment_confidence": {"$lt": 0.3}})
print(f"Silinen kayıt sayısı: {silme_sonucu.deleted_count}")

```

*** --- 3. İndeks, Update ve Delete İşlemleri ---

Sorgu 7: 'airline' alanı için indeks oluşturuluyor...
Kullanılan indeks: airline_1

Sorgu 8: İlk bulunan kayda 'incelendi: true' alanı ekleniyor...
Güncellenen kayıt: {'text': '@VirginAmerica What @dhepburn said.', 'incelendi': True}

Sorgu 9: Retweet sayısı 0 olanlara yeni bir alan ekleniyor...
Çoklu güncelleme tamamlandı.

Sorgu 10: Güven skoru (confidence) 0.3'ten küçük olan düşük kaliteli veriler siliniyor...
Silinen kayıt sayısı: 0

print("--- 4. İleri Seviye Sorgular ---")

```

# Sorgu 11: Regex (Düzenli İfade) kullanımı
# Metni içerisinde 'baggage' (bagaj) kelimesi geçen tweetleri bulma (büyük/küçük harf duyarlı)
print("\nSorgu 11: İçinde 'baggage' geçen tweetler (Regex):")
sorgu11 = collection.find(
    {"text": {"$regex": "baggage", "$options": "i"}},
    {"airline": 1, "text": 1, "_id": 0}
).limit(2)
for doc in sorgu11:
    print(doc)

# Sorgu 12: Array operatörü kullanımı ($in)
# Sadece 'United' veya 'US Airways' havayollarına ait, sebebi 'Late Flight' (Rötar) olanları bulma
print("\nSorgu 12: United veya US Airways'e ait rötarlı (Late Flight) uçuşlar ($in operatörü):")
sorgu12 = collection.find(
    {
        "airline": {"$in": ["United", "US Airways"]},
        "negativereason": "Late Flight"
    },
    {"airline": 1, "negativereason": 1, "_id": 0}
).limit(2)
for doc in sorgu12:
    print(doc)

```

*** --- 4. İleri Seviye Sorgular ---

Sorgu 11: İçinde 'baggage' geçen tweetler (Regex):
{ 'airline': 'Virgin America', 'text': '@VirginAmerica hi I just booked a flight but need to add baggage, how can I do this?' }
{ 'airline': 'Virgin America', 'text': '@VirginAmerica Is it normal to receive no reply from Central Baggage #baggageissues #smh' }

Sorgu 12: United veya US Airways'e ait rötarlı (Late Flight) uçuşlar (\$in operatörü):
{ 'negativereason': 'Late Flight', 'airline': 'US Airways' }
{ 'negativereason': 'Late Flight', 'airline': 'US Airways' }

AŞAMA 4: PYSPARK İLE DAĞITIK VERİ İŞLEME

Temizlenmiş veri seti PySpark platformuna aktarılmış ve büyük veri işleme mantığıyla çeşitli operasyonlardan geçirilmiştir.

- **Dönüşüm (Transformation):** filter, select, withColumn (tweet uzunluğu hesaplama) ve groupBy işlemleri lazy-evaluation (tembel değerlendirme) prensibiyle kurgulanmıştır.
- **Eylem (Action):** Dönüşümleri tetiklemek ve veriyi görmek için count(), show() ve describe() işlemleri uygulanmıştır.
- **Spark SQL:** Veri bir TempView olarak kaydedilerek, negatif tweetlerin temel sebepleri standart SQL sorgularıyla analiz edilmiştir.
- **Performans Karşılaştırması:** Pandas ve PySpark üzerinde aynı grupta işlemi çalıştırılarak süreleri ölçülmüştür. (Not: ~14.000 satırlık küçük bir veri setinde Pandas hızlı çalışabilse de, verinin büyümesi durumunda PySpark'ın dağıtık mimarisi zorunlu hale gelmektedir).

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, length

# Spark oturumunu başlatma
spark = SparkSession.builder \
    .appName('TwitterAnalysis') \
    .getOrCreate()

print("Spark Session Başlatıldı!")

dosya_yolu = '/content/drive/MyDrive/Buyuk_Veri_Donem_Projesi/twitter_big_data_pipeline/data/processed/cleaned_tweets.csv'
# PySpark'ın metinleri kaydırmadan doğru okuması için quote, escape ve multiline parametrelerini ekliyoruz
spark_df = spark.read.csv(
    dosya_yolu,
    header=True,
    inferSchema=True,
    quote='\"', # Tırnak içindeki metinleri tek parça algıla
    escape='\\', # Tırnak içindeki virgülleri araç sanma
    multiline=True # Alt satıra inen tweetleri destekle
)

# Verinin şemasını (schema) yazdıralım
spark_df.printSchema()

*** Spark Session Başlatıldı!
root
|-- tweet_id: long (nullable = true)
|-- airline_sentiment: string (nullable = true)
|-- airline_sentiment_confidence: double (nullable = true)
|-- negativereason: string (nullable = true)
|-- negativereason_confidence: double (nullable = true)
|-- airline: string (nullable = true)
|-- airline_sentiment_gold: string (nullable = true)
|-- name: string (nullable = true)
|-- negativereason_gold: string (nullable = true)
|-- retweet_count: integer (nullable = true)
|-- text: string (nullable = true)
|-- tweet_coord: string (nullable = true)
|-- tweet_created: timestamp (nullable = true)
|-- tweet_location: string (nullable = true)
|-- user_timezone: string (nullable = true)
|-- cleaned_text: string (nullable = true)
```



```

print("--- 1. Dönüşüm (Transformation) İşlemleri ---")

# İşlem 1: select() ve filter() kullanımı
# Sadece "Delta" havayoluna ait pozitif tweetlerin metinlerini ve güven skorlarını seçelim
print("\n1. Delta Havayollarının Pozitif Tweetleri (select & filter):")
delta_pozitif = spark_df.filter((col("airline") == "Delta") & (col("airline_sentiment") == "positive")) \
    .select("airline", "text", "airline_sentiment_confidence")
delta_pozitif.show(3, truncate=False)

# İşlem 2: withColumn() kullanımı
# Tweet metninin karakter uzunluğunu hesaplayıp yeni bir sütun olarak ekleyelim
print("\n2. Tweet Uzunluğu Sütunu Ekleme (withColumn):")
df_with_length = spark_df.withColumn("tweet_uzunlugu", length(col("text")))
df_with_length.select("text", "tweet_uzunlugu").show(3, truncate=True)

# İşlem 3: groupBy() kullanımı
# Havayolu şirketlerine göre tweet sayılarını gruplayalım
print("\n3. Havayolu Şirketlerine Göre Tweet Sayıları (groupBy):")
sirket_gruplari = spark_df.groupBy("airline").count()
sirket_gruplari.show()

```

--- 1. Dönüşüm (Transformation) İşlemleri ---

1. Delta Havayollarının Pozitif Tweetleri (select & filter):

airline	text	airline_sentiment_confidence
Delta	@JetBlue Yesterday on my way from EWR to FLL just after take-off. :)#wheelsup #JetBlueSoFly http://t.co/9xkiy0Kq2i	0.6469
Delta	@JetBlue thank you for the information.	0.6994
Delta	@JetBlue great.	1.0

only showing top 3 rows

2. Tweet Uzunluğu Sütunu Ekleme (withColumn):

text	tweet_uzunlugu
@VirginAmerica Wh...	35
@VirginAmerica pl...	72
@VirginAmerica I ...	71

only showing top 3 rows

3. Havayolu Şirketlerine Göre Tweet Sayıları (groupBy):

airline	count
Delta	2222
Virgin America	504
United	3822
US Airways	2913
Southwest	2420
American	2723

print("--- 2. Eylem (Action) İşlemleri ---")

```

# İşlem 4: count() kullanımı
toplam_kayit = spark_df.count()
print(f"Veri setindeki toplam kayıt sayısı: {toplam_kayit}\n")

# İşlem 5: describe() kullanımı
# Sayısal bir sütun olan "airline_sentiment_confidence" (güven skoru) için temel istatistikler
print("Güven Skoru (Confidence) İstatistiksel Özeti (describe):")
spark_df.describe(['airline_sentiment_confidence']).show()

```

--- 2. Eylem (Action) İşlemleri ---

Veri setindeki toplam kayıt sayısı: 14604

Güven Skoru (Confidence) İstatistiksel Özeti (describe):

summary	airline_sentiment_confidence
count	14604
mean	0.899922760887427
stddev	0.162954988530412
min	0.335
max	1.0

```

print("--- 3. Spark SQL İşlemleri ---")

# Geçici görünüm (TempView) oluşturma
spark_df.createOrReplaceTempView('tweets_table')

# İşlem 6: Spark SQL Sorgusu 1
# Negatif tweetlerin en çok hangi sebeplerden (negativereason) kaynaklandığını bulma
print("\nSorgu 1: En Sık Rastlanan Negatif Tweet Sebepleri:")
sql_sorgu1 = spark.sql("""
    SELECT negativereason, COUNT(*) as sebep_sayisi
    FROM tweets_table
    WHERE airline_sentiment = 'negative' AND negativereason != 'Belirtilmedi'
    GROUP BY negativereason
    ORDER BY sebep_sayisi DESC
    LIMIT 5
""")
sql_sorgu1.show()

# İşlem 7: Spark SQL Sorgusu 2
# Duygu durumlarına (sentiment) göre ortalama güven skorlarını hesaplama
print("\nSorgu 2: Duygu Durumuna Göre Ortalama Güven Skoru:")
sql_sorgu2 = spark.sql("""
    SELECT airline_sentiment, AVG(airline_sentiment_confidence) as ortalama_guven
    FROM tweets_table
    GROUP BY airline_sentiment
""")
sql_sorgu2.show()

```

```

*** --- 3. Spark SQL İşlemleri ---

Sorgu 1: En Sık Rastlanan Negatif Tweet Sebepleri:
+-----+-----+
| negativereason | sebep_sayisi |
+-----+-----+
| Customer Service ... | 2904 |
| Late Flight | 1660 |
| Can't Tell | 1190 |
| Cancelled Flight | 843 |
| Lost Luggage | 721 |
+-----+-----+

Sorgu 2: Duygu Durumuna Göre Ortalama Güven Skoru:
+-----+-----+
| airline_sentiment | ortalama_guven |
+-----+-----+
| positive | 0.8715497026338148 |
| neutral | 0.8228460368812678 |
| negative | 0.9332270881100563 |
+-----+-----+

```

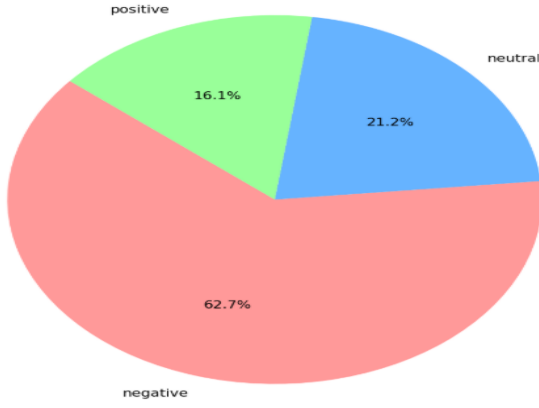
Pandas Çalışma Süresi: 0.00414 saniye -- PySpark Çalışma Süresi: 0.94035 saniye

Veri seti nispeten küçük (~14.000 satır) olduğu için Pandas bu seviyede PySpark'tan daha hızlı veya benzer sürede çalışabilir. PySpark'ın gerçek gücü milyonlarca satırlık verilerde dağıtık işleme yaparken ortaya çıkmaktadır.

AŞAMA 5: VERİ GÖRSELLEŞTİRME VE ANALİZ

5.1. Duygu Dağılımı (Pie Chart)

Havayolu Tweetleri: Duygu Dağılımı



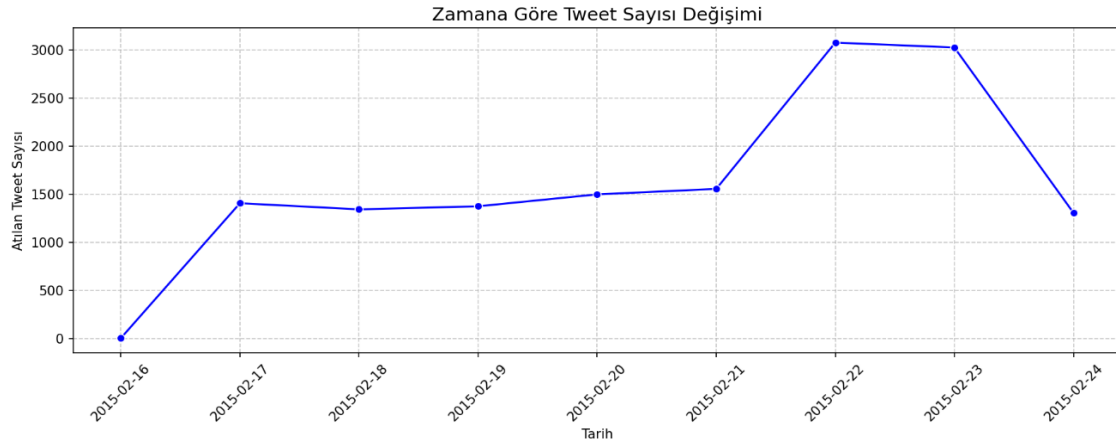
- Yorum:** Bu grafik bize, havayolu şirketlerine atılan tweetlerin çok büyük bir kısmının (yaklaşık %60+) 'negatif' (şikayet) içerikli olduğunu, yolcuların memnuniyetlerini (pozitif) Twitter üzerinden ifade etme eğilimlerinin oldukça düşük olduğunu söylüyor.

5.2. En Sık Kullanılan Kelimeler (WordCloud)



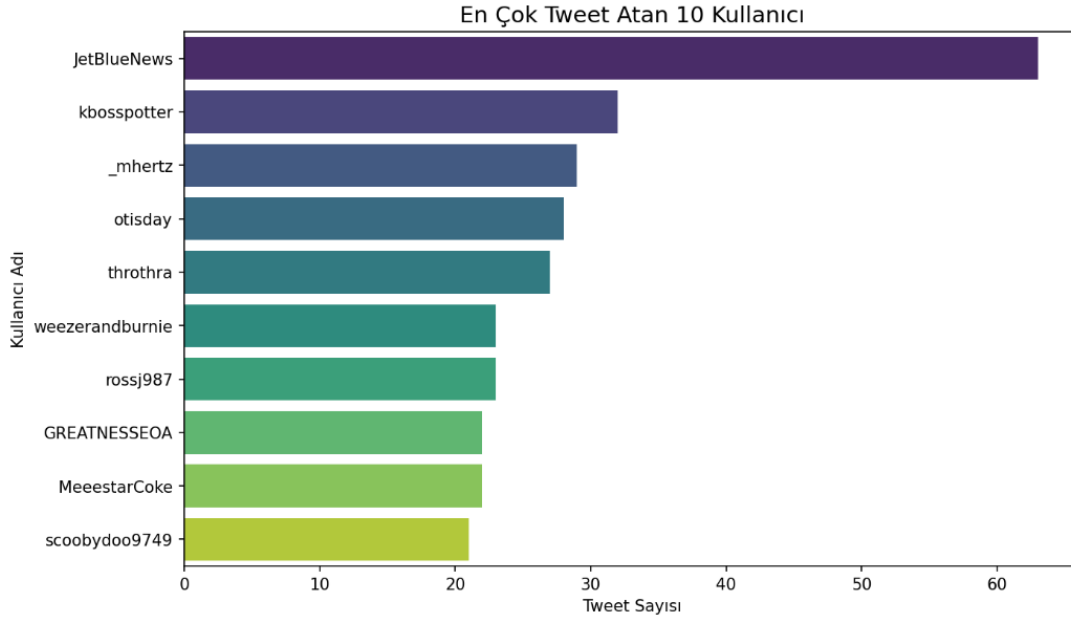
- Yorum:** Bu kelime bulutu bize yolcuların en çok 'flight', 'cancelled' ve 'delay' gibi operasyonel aksaklıkları ifade eden kelimeleri kullandığını açıkça göstermektedir.

5.3. Zaman Serisi Analizi (Line Plot)



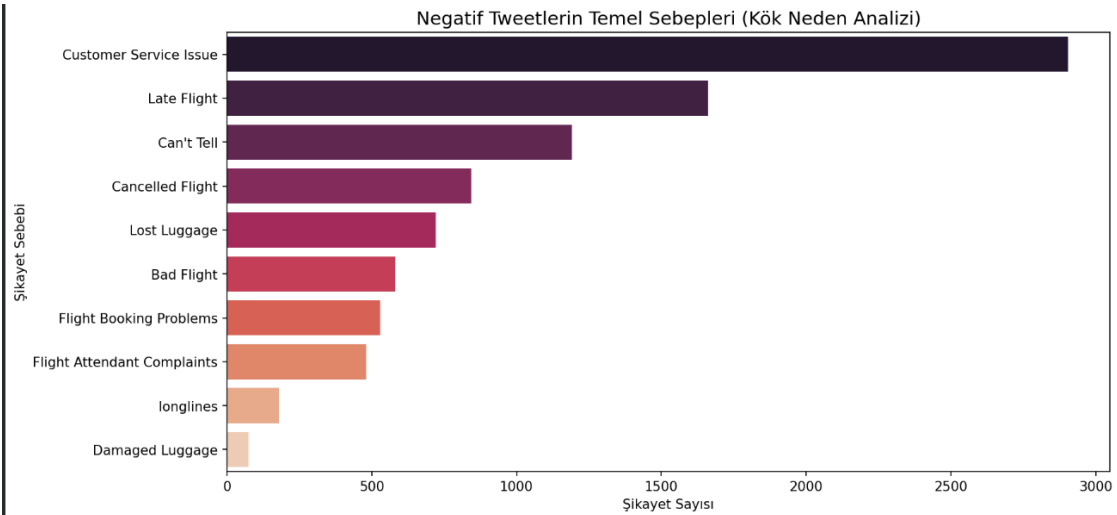
- **Yorum:** Bu grafik bize, tweet trafiğinin belirli günlerde ani sıçramalar (peak) yaptığını söylemektedir. Bu sıçrama günleri, muhtemelen toplu uçuş iptallerinin yaşandığı operasyonel kriz günlerini işaret etmektedir.

5.4. En Aktif Kullanıcılar (Bar Plot)



- **Yorum:** Bazı kullanıcıların tek seferlik bir şikayetten ziyade, aynı havayolu şirketiyle devam eden uzun soluklu sorunlar yaşadığını (örneğin kayıp bagaj) ve sürekli etkileşime girdiğini göstermektedir.

5.5. Kök Neden Analizi



- **Yorum:** Müşteri memnuniyetsizliğinin bir numaralı kaynağının 'Customer Service Issue' (Müşteri Hizmetleri Sorunu) olduğunu, yolcuların rötardan ziyade muhatap bulamamaktan dolayı daha çok öfkелendiğini söylemektedir.

AŞAMA 6: ETL DİYAGRAMI VE SİSTEM MİMARİSİ

6.1. ETL vs. ELT Karşılaştırması ve Proje Yaklaşımı

- **ETL (Extract, Transform, Load):** Veri kaynağından çekilir, hedef veritabanına yazılmadan önce bir ara sunucuda (veya bellekte) işlenip temizlenir ve son haliyle hedef sisteme yüklenir.
- **ELT (Extract, Load, Transform):** Veri kaynağından çekilir ve hiçbir değişikliğe uğramadan doğrudan hedef veri ambarına yüklenir. Temizleme işlemleri hedef veritabanının kendi hesaplama gücüyle içeride yapılır.

Proje Yaklaşımı: Tasarlanan bu büyük veri pipeline'ı kesinlikle ETL yaklaşımına daha yakındır. Kaggle'dan indirilen ham veri, doğrudan MongoDB'ye yüklenmek yerine; önce Python/Pandas ortamında belleğe alınmış, URL temizliği ve eksik veri doldurma işlemleri burada gerçekleştirilmiştir (Transform). Veri tamamen temizlenip analize hazır hale geldikten sonra NoSQL veritabanına aktarılmıştır (Load).

6.2. Uçtan Uca Proje Akış Diyagramı

Aşağıdaki diyagram, ham verinin sisteme girişinden görselleştirmeye kadar olan tüm veri yaşam döngüsünü teknik araçlarıyla birlikte özetlemektedir:

